🔍 Analyse Complète — Assistant Matanne

> **Date** : 27 mars 2026 | **Dernière mise à jour** : Sprint 5 (notifications, admin, 2FA) — 29 mars 2026
> **Auteur** : GitHub Copilot | **État global** : ~98% de couverture fonctionnelle — aucun bug critique restant

📋 Table des matières

1. Résumé exécutif
2. Architecture & inventaire technique
3. Analyse module par module
4. Bugs & régressions identifiés
5. Manques & features incomplètes
6. SQL — Consolidation & état actuel
7. Interactions intra-modules & inter-modules
8. Axes d'intégration IA
9. Jobs automatiques — Planning cron
10. Notifications — WhatsApp, Mail, Push
11. Mode Admin / Déclenchement manuel
12. Axes d'innovation identifiés
13. Synthèse & roadmap recommandée

1. Résumé exécutif

L'application en chiffres

Indicateur	Valeur
Modules fonctionnels	8 (Cuisine, Famille, Maison, Jeux, Planning, Outils, Admin, Dashboard)
Endpoints API	~350+ réels (aucun 501 détecté)
Pages frontend	~60 pages Next.js (App Router)
Tables SQL	~130 tables
Services backend	~200+ fichiers Python dans `src/services/`
Modèles ORM	128+ classes SQLAlchemy dans 28 fichiers
Tests	197 fichiers (141 backend + 46 frontend unit + 10 E2E Playwright)
Couverture phases	26/28 complètes, 2 quasi-complètes, 0 partielle
Cron jobs APScheduler	10 jobs actifs (était 6)
Canaux de notification	4 (ntfy.sh, Web Push VAPID, Email Resend, WhatsApp webhook)
WebSockets	4 channels (courses, planning, notes, projets)

Points forts

- ✅ **Architecture robuste** : FastAPI + SQLAlchemy 2.0 + Pydantic v2, cache multi-niveaux, circuit breaker, rate limiting
- ✅ **IA bien intégrée** : Mistral (texte) + Pixtral (vision) présents dans 15+ services

- ✅ **Modules Famille, Jeux, Maison** : 100% fonctionnels avec IA contextuelle
- ✅ **Infrastructure prod-ready** : Sentry, Prometheus, CI/CD 8 workflows, PWA offline, 2FA TOTP, RGPD, WebSockets

✅ Tous les bugs critiques résolus (Sprint 5)

- ✅ **Push subscriptions persistées en DB** : `NotificationPersistenceMixin` dans `notif_web_persistence.py` → les abonnements VAPID survivent aux redémarrages
- ✅ **Canal email opérationnel** : `ServiceEmail` via Resend (`notif_email.py`) — reset password, verify email, résumé hebdo, alertes critiques
- ✅ **Digest ntfy + rappel courses schedulés** : jobs `digest_ntfy` (09h00) et `rappel_courses` (18h00) ajoutés à APScheduler
- ✅ **WhatsApp webhook entrant** : `webhooks_whatsapp.py` — receive + réponse IA via Meta Cloud API
- ✅ **Dispatcher multi-canal** : `DispatcherNotifications` (`notif_dispatcher.py`) — ntfy / push / email via `envoyer(user_id, message, canaux=[…])`

Points forts renforcés (Sprint 5)

- ✅ **2FA TOTP complet** : `/2fa/enable` , `/2fa/verify-setup` , `/2fa/disable` , `/2fa/status` , `/2fa/login` — `ServiceDeuxFacteurs` (125 lignes)
- ✅ **Admin enrichi** : `admin.py` (425 lignes) — audit logs, triggers manuels de jobs, test notif, purge cache, liste utilisateurs
- ✅ **Version Jules recette** : bouton 🏷️ sur fiche recette → `POST /recettes/{id}/version-jules` + `ServiceVersionRecetteJules`
- ✅ **Profil aliments exclus Jules** : `GET/PUT /famille/jules/aliments-exclus` + coaching hebdo `GET /famille/jules/coaching-hebdo`
- ✅ **RouteurIA** : `core/ai/router.py` (704 lignes) — routage multi-fournisseur avec classificateur de complexité (`_ClassifieurComplexite`)
- ✅ **Cache Redis L2** : `CacheRedis` dans `caching/redis.py` (optionnel, en complement du L1+L3)

2. Architecture & inventaire technique

Stack technologique

```
Backend
├── Python 3.13+ / FastAPI
├── SQLAlchemy 2.0 ORM + Pydantic v2
├── Mistral AI (texte) + Pixtral (vision) + RouteurIA multi-fournisseur
├── APScheduler (cron) — 10 jobs actifs
├── ntfy.sh + Web Push VAPID + Email (Resend) + WhatsApp webhook
├── Redis (L2 cache optionnel)
└── Supabase PostgreSQL (production)

Frontend
├── Next.js 16.2.1 (App Router + Turbopack)
├── TypeScript 5 + Tailwind CSS v4 + shadcn/ui
├── React 19.2.4 + TanStack Query v5.94.5 (data fetching + cache)
├── Zustand 5.0.12 (state global)
├── React Hook Form 7.72 + Zod 4.3.6 (validation)
├── @react-three/fiber 9.5 + Three.js 0.183 (visualisation 3D maison)
└── PWA + IndexedDB (offline)

Infrastructure
├── 8 GitHub Actions workflows
├── Dockerfile + Docker Compose staging
├── Sentry (FE + BE) — conditionnel avec DSN
├── Prometheus + métriques custom
└── Alembic (migrations — 3 fichiers à absorber dans INIT_COMPLET)
```

WebSockets actifs

Channel	Path	Usage
Courses	`/ws/courses`	Collaboration temps réel listes de courses
Planning	`/ws/planning`	Sync planning semaine multi-utilisateurs
Notes	`/ws/notes`	Édition collaborative de notes
Projets	`/ws/projets`	Suivi projets maison en temps réel

Cron jobs existants

Job ID	Schedule	Action
`rappels_famille`	07h00 quotidien	Anniversaires, documents, crèche, jalons
`rappels_maison`	08h00 quotidien	Garanties, contrats, entretien
`rappels_generaux`	08h30 quotidien	Inventaire, garanties générales
`entretien_saisonnier`	Lundi 06h00	Création tâches entretien saisonnières
`resume_hebdo`	Lundi 07h30	Résumé hebdo famille complet
`push_quotidien`	09h00 quotidien	Web Push alertes urgentes + jeux
`digest_ntfy`	09h00 quotidien	Digest ntfy.sh quotidien (tâches + rappels)
`rappel_courses`	18h00 quotidien	Alerte ntfy.sh articles courses en attente
`push_contextuel_soir`	18h00 quotidien	Push contextuel soir (planning + météo)
`enrichissement_catalogues`	1er du mois 03h00	Enrichissement IA 4 catalogues JSON

3. Analyse module par module

🍽️ Cuisine

État : ~93% complet | Backend très solide, quelques features UI restantes

Ce qui est implémenté ✅

- CRUD recettes complet (import URL, PDF, enrichissement auto nutrition/bio/robots)
- Filtres appareils sur liste recettes (Cookeo / Monsieur Cuisine / Air Fryer)
- `VersionRecette` ORM + service (`type_version: "jules" | "batch_cooking"`) — base pour l'adaptation menu Jules
- Planning semaine IA (Mistral) avec suggestions contextuelles météo
- `dessert_jules_recette_id` + `entree_recette_id` dans le modèle `Repas` (lignes de planning séparées)
- Listes de courses + génération depuis planning + collaboration WebSocket
- Suggestions bio-local + articles récurrents suggérés dans courses
- Idempotency cache sur l'API courses (évite les doublons de soumission)
- Inventaire avec badges Open Food Facts (Nutri-Score, Éco-Score, NOVA)
- ScanneurMultiCodes + EtiquetteQR (scan de lots + étiquettes QR de stockage)
- Photo frigo OCR Pixtral multi-zones (frigo / placard / congélateur)
- Anti-gaspillage : score hebdo + historique + trophées + suggestions IA + service zéro déchet batch
- Batch cooking : sessions, étapes par robot, préparations surgelées, `avec_jules` (tâches adaptées à Jules)
- Stepper "Ma Semaine" 4 étapes (Planning → Inventaire → Courses → Récap)
- Scan ticket de caisse → import courses avec sélection article par article (OCR Pixtral)

- Import PDF recette avec enrichissement (calories, tags, robots)
- Page /cuisine/nutrition : histogramme quotidien + KPIs macros
- BadgeNutriscore + ConvertisseurInline embarqués dans la fiche recette et le planning
- 🍋 **Bouton Version Jules** : `POST /api/v1/recettes/{id}/version-jules` → `ServiceVersionRecetteJules` (171 lignes) crée une `VersionRecette` avec substitutions IA (sel → retirer, épices → retirer, alcool → fond de volaille, saumon fumé → saumon cuit)
- 🎲 **Recette Surprise** : `GET /api/v1/recettes/surprise` — recette aléatoire filtrée saison/frigo
- Import recette depuis photo plat (`POST /generer-depuis-photo`) — Pixtral

Ce qui manque ❌

- **OCR photo-frigo → sync inventaire automatique** : le bouton "ajouter à l'inventaire" existe mais l'auto-sync continu n'est pas implémenté
- **Onboarding "Ma Semaine"** : popup première utilisation guidant l'utilisateur sur le stepper
- **Table nutrition incomplète** : 47 ingrédients couverts sur ~200+ ciblés → calcul macro silencieusement incomplet pour beaucoup de recettes
- **Rappels jour-par-jour cuisine** : push quotidien "ce soir tu prépares X" basé sur planning (job J-02 prévu)
- **Saisonnalité enrichie inventaire** : affichage des produits de saison disponibles au marché + préférences sources d'approvisionnement (Biocoop / marché / ferme)

Bugs ⚠️

- `RetourRecette` utilise `feedback='like'|'neutral'` pour les favoris, mais les types TypeScript frontend utilisent `est_favori: boolean` → potentielle désynchronisation si le champ est lu directement
- `url_source` absent du modèle ORM `Recette` mais attendu dans `RetetteResponse` (retourne `None` systématiquement)

👨‍👩‍👧 Famille

État : ~98% complet | Module le plus avancé

Ce qui est implémenté ✅

- Hub famille contextuel (Aujourd'hui/À venir/Suggestions IA)
- Suivi Jules : jalons de développement, carnet de santé numérique (vaccinations, RDV pédiatre, notes santé)
- Activités : CRUD + suggestions IA auto-préfill + météo contexte
- Routines : matin/soir avec complétion rapide et streak
- Budget : CRUD + analyse IA anomalies + prédictions + OCR tickets
- Achats famille : onglets par personne, scoring IA, suggestions Vinted, annonces LBC générées par IA
- Anniversaires : CRUD + checklist auto générée (invitations, gâteau, cadeaux...)
- Config Garde : zones, semaines fermeture crèche, jours-sans-crèche planning
- Weekend suggestions IA : activités, météo, séjours
- Contacts, Documents, Calendriers (Google OAuth)
- **Profil aliments exclus Jules** : `GET/PUT /api/v1/famille/jules/aliments-exclus` — liste configurable transmise à l'IA lors de la génération des versions Jules (sel, miel avant 12 mois, épices, alcool, charcuterie fumée, viande crue)
- **Coaching hebdo Jules** : `GET /api/v1/famille/jules/coaching-hebdo` — `JulesAIService` analyse la semaine (activités, repas, carnet santé) → message hebdo contextuel
- **Version Jules recette** : bouton 🍋 déclenche `POST /recettes/{id}/version-jules` depuis la fiche Jules (lien depuis section Cuisine)

Ce qui manque ❌

- **Tests famille** : `tests/api/test_famille_achats.py` et `test_famille_garde.py` marqués "À FAIRE" dans le roadmap (CT-07)
- **Rappels médicaux Jules** : vaccins (calendrier vaccinal standard) + RDV pédiatre en push ntfy/Web Push
- **Deep link Family Album** : bouton "📸 Partager sur Family Album" sur les fiches activités/anniversaires Jules (app iOS externe, pas d'API — deep link uniquement)

🏡 Maison

État : ~95% complet | Très riche, seul jardin IA enrichi manquant

Ce qui est implémenté ✅

- Hub contextuel maison (briefing quotidien + alertes actives)
- 152 endpoints : projets, routines, entretien prédictif, jardin, stocks cellier, meubles, artisans, contrats, garanties (+prédictions panne IA), diagnostics, estimations, eco-conseils, devis, visualisation pièces
- Ménage : planning hebdomadaire IA + tâches ponctuelles
- Énergie : releves, tendances, prévisions IA, conseils économies
- Artisan assistant IA (chat maison)
- Gamification entretien (points, badges)
- Visualisation pièces avec objets et historique

Ce qui manque ❌

- **Suggestions jardin IA enrichies** (Phase AB) : au-delà des règles catalogue, pas de génération IA contextuelle saisonnière
- **Cellier ↔ inventaire cuisine** : pas de synchronisation entre le cellier maison et l'inventaire cuisine (doublons potentiels)
- **Budget ménager global** : pas de synthèse finances maison + famille sur un même tableau
- **Maintenance prédictive étendue** : prédictions panne sur garanties ✅, mais pas sur objets maison non garantis

🎮 Jeux

État : 100% complet ✅

Ce qui est implémenté ✅

- Dashboard value-bets + KPIs bankroll
- Paris sportifs : CRUD + stats + prédictions IA + analyse patterns + value-bets
- Loto + Euromillions : tirages, grilles, stats, génération IA pondérée, numéros en retard
- Performance : ROI détaillé + breakdown par sport/championnat
- Jeu responsable : suivi mensuel, limites, auto-exclusion
- OCR tickets loto (Pixtral)
- Historique des cotes
- Séries de victoires/défaites avec alertes

Ce qui manque ❌

- **Actualisation cotes à la demande** : les cotes sont saisies manuellement — prévoir un bouton "Actualiser" sur la fiche match qui interroge une API bookmaker (Oddsportal, The Odds API…) uniquement à la consultation, pas de scraping continu
- **Statistiques avancées par équipe** : possession, xG, forme à domicile/extérieur (données football.api)

🗓️ Planning général

État : ~80% complet

Ce qui est implémenté ✅

- Planning repas semaine + génération IA
- Nutrition hebdo (macros par jour)
- Export iCal
- "Ma Semaine" unifiée (repas + activités + tâches maison)
- Timeline page

Ce qui manque ❌

- **Vue calendrier mensuelle aggregée** : tous les événements (famille + maison + jeux) sur un même calendrier
- **Rappels push cuisine** : "Ce soir : Poulet rôti (45min de préparation)" → non implémenté
- **Conflits de planning** : `ServiceConflits` existe mais non exposé en frontend
- **Intégration Google Calendar bidirectionnelle** : OAuth ✅, mais sync des événements Planning vers Google non testé de bout en bout

🎯 Dashboard

État : ~85% complet

Ce qui est implémenté ✅

- Widgets : Stock critique, Repas du jour, Prochaines activités, Budget famille, Jeux performances, Rappels
- Actions rapides contextuelles
- `GET /api/v1/dashboard/cuisine` : données cuisine agrégées

Ce qui manque ❌

- **Widgets configurables** : drag & drop, choix des métriques affichées (roadmap ouvert)
- **Score de santé global** : indicateur synthétique (bien-être famille + maison + budget)
- **Météo intégrée au dashboard** : la météo est dans `/outils/meteo` mais pas dans le dashboard principal

🛠️ Outils

État : ~90% complet

Ce qui est implémenté ✅

- Chat IA Mistral multi-contexte (cuisine, maison, famille, jeux, général)
- Convertisseur unités (masse, volume, températures)
- Minuteur multi (cuisine)
- Notes + Journal + Contacts + Liens favoris + Mots de passe maison
- Météo (Open-Meteo)
- Nutritionniste IA : histogramme hebdo + chat Mistral nutrition + KPIs macros

Ce qui manque ❌

- **Partage de notes** : les notes sont privées, pas de partage famille
- **Notes vocales** : dictaphone → texte (Web Speech API ou Whisper)
- **Outils transversaux** : les outils sont embarqués dans les modules qui en ont besoin — convertisseur dans inventaire et courses, minuteur déclenché depuis le temps de cuisson d'une recette, météo dans le dashboard + jardin + famille, chat IA accessible depuis n'importe quel module. La page `/outils` reste l'accès direct centralisé

🔐 Admin

État : ~90% complet ✅ | Sprint 5 — panneau admin complètement enrichi

Ce qui est implémenté ✅

- Page admin `/admin` protégée par rôle
- Audit logs : liste, stats, export CSV (`GET /admin/audit-logs` , `/audit-stats` , `/audit-export-csv`)

- **Jobs APScheduler** : `GET /admin/jobs` (liste tous les jobs + dernier run + statut), `POST /admin/jobs/{job_id}/run` (déclenchement immédiat manuel)
- **Test notifications** : `POST /admin/notifications/test` — envoie une notif test sur le canal demandé (ntfy/push/email)
- **Cache** : `POST /admin/cache/purge` + `GET /admin/cache/stats` (hits/misses par module)
- **Utilisateurs** : `GET /admin/users` (liste des comptes + rôles + dernier login)
- **2FA TOTP** : `ServiceDeuxFacteurs` (125 lignes) — `/2fa/enable`, `/2fa/verify-setup`, `/2fa/disable`, `/2fa/status`, `/2fa/login`
- **ServiceAudit** (354 lignes) : traçage de toutes les actions admin en base
- Backup DB : CLI script + GitHub Actions workflow

Ce qui manque ❌

- **Tableau de bord monitoring** : métriques Prometheus exposées mais pas visualisées dans l'UI admin (graphes)
- **Modification/suppression utilisateurs** depuis l'UI admin (liste disponible via `GET /admin/users`, mais PATCH/DELETE non exposés encore)
- **Page frontend jobs** : `frontend/src/app/(app)/admin/jobs/page.tsx` non créée — les endpoints backend existent mais pas l'UI de déclenchement

4. Bugs & régressions identifiés

✅ Critiques résolus (Sprint 5)

#	Bug	Fix	Statut
B-01	Push subscriptions perdues au redémarrage	`NotificationPersistenceMixin` dans `notif_web_persistence.py` (317 lignes) — abonnements persistés en DB (`abonnements_push_vapid`)	✅ CORRIGÉ
B-02	Pas d'endpoint VAPID public key	`GET /api/v1/push/vapid-public-key` ajouté dans `push.py`	✅ CORRIGÉ
B-03	Digest ntfy + rappel courses jamais schedulés	Jobs `digest_ntfy` (09h00) et `rappel_courses` (18h00) ajoutés à APScheduler	✅ CORRIGÉ
B-05	Accès direct `_subscriptions` dans le cron	`NotificationPersistenceMixin` remplace le dict en mémoire	✅ CORRIGÉ

🟠 Hauts

#	Bug	Localisation	Impact
B-04	**Cache L1 non partagé** : en mode multi-workers (Uvicorn + Gunicorn), chaque process a son propre L1. Sans Redis configuré, les workers se désynchronisent	`src/core/caching/orchestrator.py`	Données périmées affichées selon le worker
B-06	`url_source` **absent du modèle Recette** : le schéma `RecetteResponse` retourne toujours `None` pour `url_source` mais l'UI attend une URL cliquable	`src/core/models/recettes.py`, `src/api/schemas/recettes.py`	Lien "source" non fonctionnel sur les recettes importées depuis URL
B-07	`verifier_saison` **silencieusement ignoré** : le cron entretien saisonnier vérifie `hasattr()` mais ne log qu'en DEBUG si absent → aucun feedback si le service ne démarre pas	`src/services/core/cron/jobs.py`	Maintenance saisonnière silencieusement inactive
B-08	`RepasPlanning` **modèle manquant** : 1 test dashboard est `skip` car le modèle n'est pas défini	`tests/api/test_routes_dashboard_jeux.py`	Couverture test incomplète

🟡 Moyens

#	Bug	Localisation	Impact
B-09	**Favicon/manifest Push** : le `manifest.json` ne définit pas les champs `gcm_sender_id` / `background_sync` nécessaires pour certains navigateurs	`frontend/public/manifest.json`	Comportement push instable sur Chrome Android
B-10	**Table `liste_cours` en doublon** : SQL contient à la fois `liste_cours` et `listes_courses` (coquille)	`sql/INIT_COMPLET.sql`	Confusion dans les requêtes directes SQL
B-11	**Album/Journal frontend subsistent** : les pages effectuent des `redirect()` mais le lien sidebar existe encore pour "Calendriers" qui pointe vers `/famille/calendriers` inexistant	`frontend/src/composants/disposition/barre-laterale.tsx`	404 potentiel
B-12	**`RetourRecette.est_favori` vs `feedback`** : ORM utilise `feedback='like'`, les types TypeScript utilisent `est_favori: boolean`	`frontend/src/types/recettes.ts`	Désync si la propriété est lue directement

🟢 Mineurs / Cosmétiques

#	Bug	Localisation
B-13	**Nutrition table 47/200+ ingrédients** : calcul macro silencieusement incomplet	`data/reference/nutrition_table.json`
B-14	**L3 cache jamais écrit** : `persistent=False` par défaut → L3 lecture seule seulement	`src/core/caching/orchestrator.py`
B-15	**Playwright tests CI** : uniquement `--project=chromium`, aucun test Firefox/WebKit en CI	`.github/workflows/frontend-tests.yml`
B-16	**Sprint11 connu** : `asyncio_mode=strict` requis, `pytestmark` manquant dans certains nouveaux fichiers de test	`pytest.ini`, tests récents

5. Manques & features incomplètes

🍽️ Cuisine — lacunes

Feature	Effort	Impact
OCR photo-frigo → auto-ajout inventaire sans confirmation	S	⭐⭐⭐
Rappels push "soir cuisine" : "Ce soir tu prépares X" (job J-02)	M	⭐⭐⭐
Table nutrition étendue (200+ ingrédients vs 47 actuels)	M	⭐⭐
Onboarding Ma Semaine (popup first-use)	S	⭐⭐
Partage de liste de courses avec invité (lien ou QR code)	M	⭐⭐
Sources approvisionnement préférées (Biocoop / marché / ferme)	S	⭐⭐
Scan code-barres continu (stream caméra) pour inventaire	L	⭐⭐

👨‍👩‍👧 Famille — lacunes

Feature	Effort	Impact
Tests famille (achats, garde)	M	⭐⭐
Rappels médicaux Jules (vaccins, RDV pédiatre) en push	S	⭐⭐⭐
Deep link Family Album pour événements Jules	XS	⭐⭐

🏡 Maison — lacunes

Feature	Effort	Impact
Cellier ↔ Inventaire cuisine : synchronisation	M	⭐⭐⭐
Budget ménager global : vue agrégée maison + famille	M	⭐⭐
Suggestions jardin IA saisonnières enrichies	S	⭐⭐
Comparateur énergie (historique vs voisins ou indices nationaux)	L	⭐⭐

📊 Dashboard — lacunes

Feature	Effort	Impact
Widgets drag & drop configurables	L	⭐⭐⭐
Score de bien-être synthétique (Famille + Maison + Budget)	M	⭐⭐⭐
Météo intégrée (widget dashboard)	S	⭐⭐
Activité récente / fil d'événements (timeline)	M	⭐⭐

🔔 Notifications — lacunes résiduelles

Feature	Effort	Statut
~~Persistance Push subscriptions en DB~~	~~S~~	✅ FAIT (Sprint 5)
~~Endpoint VAPID public key~~	~~XS~~	✅ FAIT (Sprint 5)
~~Canal email~~	~~L~~	✅ FAIT (Sprint 5 — Resend)
~~ntfy digest quotidien schedulé~~	~~XS~~	✅ FAIT (Sprint 5)
~~Rappel courses ntfy~~	~~XS~~	✅ FAIT (Sprint 5)
Canal SMS	L	⭐ Optionnel
Dispatcher WhatsApp sortant (messages proactifs)	M	⭐⭐ (webhook entrant ✅, sortant non)

6. SQL — Consolidation & état actuel

État du schéma

Le fichier `sql/INIT_COMPLET.sql` (v3.0, 4416 lignes) est **l'unique source de vérité** en mode développement. Il est **idempotent** (`DROP TABLE ... CASCADE` en tête) et peut être ré-exécuté sans danger dans Supabase SQL Editor.

Structure du schéma

```
sql/
├── INIT_COMPLET.sql     ← Schéma complet + RLS + indexes + triggers + inline migrations
└── migrations/          ← Fichiers de migration SQL (dev, non utilisés actuellement)
```

Incohérences et points à consolider

#	Problème	Tables concernées	Action recommandée
SQL-01	**Doublon `liste_cours` vs `listes_courses`**	`liste_cours` , `listes_courses`	Supprimer `liste_cours` (coquille) et vérifier tous les `INSERT INTO liste_cours`
SQL-02	**Colonnes `user_id` manquantes**	Certaines tables de liaison N:M n'ont pas de `user_id` → RLS trop permissif	Auditer les tables sans `user_id` + RLS
SQL-03	**Tables orphelines sans modèle ORM**	`journal_bord` , `sessions_travail` , `log_statut_objets` (à vérifier)	Mapper via ORM ou supprimer
SQL-04	**Tables ORM sans SQL**	`voyage.py` , `album.py` (supprimé) modèles ORM restants sans `CREATE TABLE` dans INIT_COMPLET	Nettoyer les deux sens
SQL-05	**Inline migrations en bas de fichier**	`ALTER TABLE recettes ADD COLUMN ...` en bas de INIT_COMPLET	Déplacer en tête dans les sections `CREATE TABLE` correspondantes
SQL-06	**Index manquants**	`recette_ingredients(ingredient_id)` , `repas(planning_id)` , `articles_modeles(modele_id)`	Ajouter pour les JOINs les plus fréquents
SQL-07	**Contraintes FK non nommées**	`REFERENCES users(id)` sans `CONSTRAINT fk_xxx`	Nommer pour faciliter les erreurs de debug
SQL-08	**Pas de partition temporelle**	`historique_inventaire` , `historique_action` , `audit_logs` grossissent indéfiniment	Envisager partition par année ou TTL trigger

Recommandation SQL (mode dev)

Puisque tu es en dev et que tu ne veux pas de migrations, voici le processus recommandé :

```
# Pour ajouter une colonne :
# 1. Modifier src/core/models/xxx.py (ajout mapped_column)
# 2. Modifier sql/INIT_COMPLET.sql à l'endroit exact du CREATE TABLE
# 3. Ré-exécuter INIT_COMPLET.sql dans Supabase SQL Editor (idempotent)
# 4. Commit une seule fois les 2 changements ensemble

# Vérification rapide de cohérence ORM ↔ SQL :
grep -h "__tablename__" src/core/models/*.py | sort > /tmp/orm_tables.txt
grep "^CREATE TABLE" sql/INIT_COMPLET.sql | sort > /tmp/sql_tables.txt
diff /tmp/orm_tables.txt /tmp/sql_tables.txt
```

Plan de consolidation SQL (à faire — P-06/P-07)

Objectif : `sql/INIT_COMPLET.sql` **= seule source de vérité absolue** — plus aucun `ALTER TABLE` flottant ni fichier de migration séparé.

Fichiers à absorber puis supprimer

Fichier	Contenu à intégrer dans INIT_COMPLET	Action
`sql/migrations/001_routine_moment_journee.sql`	`moment_journee VARCHAR(20)` + `jour_semaine INTEGER` sur `routines`	Déplacer dans le `CREATE TABLE routines`

Fichier	Contenu à intégrer dans INIT_COMPLET	Action
`sql/migrations/002_standardize_user_id_uuid.sql`	Colonnes `user_id VARCHAR → UUID` sur 5 tables (`preferences_utilisateurs` , `historique_actions` , `etats_persistants` , `configs_calendriers_externes` , `retours_recettes`)	Corriger les types directement dans chaque `CREATE TABLE`
`sql/migrations/003_add_cotes_historique.sql`	Table `jeux_cotes_historique` (cotes bookmaker par match + timestamps + RLS)	Ajouter comme bloc `CREATE TABLE` dans la section Jeux
`alembic/versions/f7e8d9c0b1a2_famille_refonte_phase1.py`	Colonnes `heure_debut TIME` (activites), `derniere_completion DATE` (routines), `pour_qui/a_revendre/prix_revente_estime/vendu_le` (achats), 8 colonnes JSONB (preferences_utilisateurs)	Intégrer dans les `CREATE TABLE` concernés
`alembic/versions/c8d1e2f3a4b5_anniversaire_checklists.py`	Tables `checklists_anniversaire` + `items_checklist_anniversaire` + 7 index	Ajouter les 2 blocs `CREATE TABLE` avec index
`alembic/versions/a1b2c3d4e5f6_initial_baseline.py`	Baseline vide (`pass`) — aucun DDL	Supprimer directement

Une fois absorbé : supprimer `sql/migrations/001|002|003.sql` + les 3 fichiers `alembic/versions/` + archiver `alembic.ini` si Alembic n'est plus utilisé.

Tables à valider dans INIT_COMPLET après absorption

- `routines` : colonnes `moment_journee` , `jour_semaine` , `derniere_completion` présentes ?
- `activites_famille` : colonne `heure_debut TIME` présente ?
- `achats_famille` : colonnes `pour_qui` , `a_revendre` , `prix_revente_estime` , `vendu_le` présentes ?
- `preferences_utilisateurs` : `user_id UUID` (pas `VARCHAR`) + 8 colonnes JSONB (`taille_vetements_*` , `style_achats_*` , `interets_*` , `equipement_activites` , `config_garde`) présentes ?
- `jeux_cotes_historique` : table complète avec 3 index + RLS présente ?
- `checklists_anniversaire` + `items_checklist_anniversaire` : 2 blocs `CREATE TABLE` + 7 index présents ?

7. Interactions intra-modules & inter-modules

Interactions existantes (déjà câblées)

```
Planning ↔ Courses       : generer_depuis_planning (liste auto depuis menu de la semaine)
Planning ↔ Inventaire    : suggestions tiennent compte du stock disponible
Planning ↔ Batch Cooking : générer session batch depuis un planning
Inventaire ↔ Anti-gaspi  : produits proches péremption → suggestions recettes
Photo-frigo ↔ Suggestions : image → identification → recettes avec ingrédients détectés
Famille/Budget ↔ Dashboard: budget disponible → widget dashboard
Jeux/Bankroll ↔ Jeux/Responsable : limites financières → alertes paris
Maison/Énergie ↔ Maison/Finances : relèves → dépenses énergie
Famille/Config ↔ Planning : jours sans crèche → planning famille
```

Interactions à créer (haute valeur)

🔗 Cellier ↔ Inventaire Cuisine

Problème : Le cellier (`ArticleCellier`) et l'inventaire cuisine (`ArticleInventaire`) sont deux silos séparés avec des données similaires (produits alimentaires, quantités).

Solution : Ajouter un champ `source: "cellier"|"cuisine"` dans la vue unifiée, ou créer un endpoint `/api/v1/inventaire/consolide` qui merge les deux.
Bénéfice : Une seule liste de courses générée tenant compte de tout le stock maison.

🔗 Anti-gaspillage ↔ Planning

Problème : La semaine suivante est plannifiée sans regard aux produits qui périment.
Solution : Ajouter un step "Urgences anti-gaspi" dans le stepper Ma Semaine — propose d'inclure les recettes qui useront les produits à risque.
API : `POST /api/v1/planning/generer?inclure_antigaspi=true`

🔗 Jeux/Bankroll ↔ Famille/Budget

Problème : Les pertes/gains de paris ne sont pas reflétés dans le budget familial.
Solution : Un bouton "Importer résultat du mois" dans Budget Famille qui additionne `somme_pertes_mois` des paris en `ArticleBudget` catégorie "loisirs/jeux".
API : `GET /api/v1/famille/budget/import-jeux-mois`

🔗 Jules ↔ Planning Cuisine (Version Jules)

Concept : Jules mange comme nous, mais avec des substitutions adaptées à son âge. L'infrastructure est déjà en place : `VersionRecette(type_version="jules")` + champ JSONB `ingredients_modifies`.
Solution : Sur chaque fiche recette, bouton 🍼 "Version Jules" → appel Mistral avec le profil aliments exclus Jules (sel, épices, alcool, saumon fumé → saumon cuit, viande crue → cuisson obligatoire…) → création `VersionRecette` sauvegardée et réutilisable.
Lien planning : Le champ `dessert_jules_recette_id` dans `Repas` permet déjà de mettre une version Jules différente pour le dessert. À étendre au plat principal.
API : `POST /api/v1/recettes/{id}/version-jules` → `VersionRecette`.

🔗 Maison/Entretien ↔ Routines

Problème : Les routines ménagères et les tâches d'entretien sont dans deux silos différents.
Solution : Vue unifiée "À faire aujourd'hui" proposée sur les jours configurables : télétravail, jours de congé, ou tout jour non-travaillé. Configurable dans `/parametres` — pas en dur le weekend. Fusionnerait routines en retard + tâches entretien urgentes + tâches jardin.
API : `GET /api/v1/maison/planning-jour?date=...`

🔗 Moteur d'alertes météo contextuel (cross-modules)

Concept : La météo influence simultanément plusieurs modules. Un moteur d'alertes centralisé produit des actions contextuelles selon les conditions, qui remontent dans le dashboard + les notifications.

Météo	Jardin	Cuisine	Famille (Jules)	Maison/Extérieur
❄️ Gel (<0°C)	Voile hivernage, rentrer les pots	Décongeler viande la veille	Manteau Jules	Rentrer le salon de jardin
🌡️ Canicule (>32°C)	Arroser matin + soir	Recettes fraîches + proposer glaces dans le planning 🍦	Hydratation Jules, pas de sortie 12h-16h	Fermer volets dès le matin
💨 Vent fort	—	—	—	Rentrer parasol, meubles extérieurs, déco
🌧️ Pluie prolongée	Stopper arrosage automatique	—	Activités intérieures Jules	—
🌸 Pollen élevé	—	—	Alerte si allergie notée dans profil famille	—

Note : les alertes s'appliquent à **tous les membres** du profil famille dont une allergie est notée (pas uniquement Jules). Les seuils (0°C, 32°C…) et les réponses sont configurables dans `/parametres/famille`.
Déjà partiel : `suggestions-rapides` cuisine utilise la météo. L'API météo Open-Meteo est déjà intégrée dans `/outils/meteo`.
API : `GET /api/v1/dashboard/alertes-contextuelles` (météo + date du jour → liste d'alertes cross-modules).

🔗 Planificateur d'événements ↔ Budget Famille ↔ Courses

Concept : Étendre au-delà des anniversaires — couvrir tous les événements familiaux avec checklist et budget adapté.
Types d'événements : 🎂 Anniversaire (Jules / famille / ami), ⛪ Baptême, 💍 Mariage, 🥖 Barbecue / Repas amis, 🎉 Fête / Soirée
Solution : Depuis une fiche événement, générer en 1 clic : checklist IA adaptée au type (événement, invités, lieu, courses proportionnelles aux convives, budget prévisionnel) + événement calendrier.
API : `POST /api/v1/famille/evenements/{id}/preparer` (généralisation de l'endpoint anniversaire existant)

🔗 Recherche globale ↔ Tous modules

Existant : `/api/v1/recherche/global` cherche recettes, projets, activités, notes, contacts.
Manquant : Recettes dans planning, garanties, artisans, documents famille absents de la recherche.
Solution : Étendre le handler de recherche avec 5 requêtes parallèles supplémentaires.

🔗 Export PDF ↔ Tous modules

Existant : Export PDF courses, planning, recettes, budget.
Manquant : Export "Rapport mensuel famille" (activités Jules + budget + anniversaires du mois) et "Carnet de bord maison" (entretiens, dépenses, projets).

8. Axes d'intégration IA

IA déjà intégrée (ne pas dupliquer)

Module	Service IA	Modèle
Planning repas	`ServiceSuggestions.suggerer_menus()`	Mistral Large
Photo frigo	`MultiModalAIService`	Pixtral 12B (vision)
Batch cooking	`ServiceBatchCooking.generer_planning_robot()`	Mistral Large
Anti-gaspillage	`ServiceSuggestions.anti_gaspillage()`	Mistral Large
Budget famille	`AnalyseBudgetIA` + anomalies	Mistral Large
Activités Jules	`JulesAIService`	Mistral Large
Weekend	`WeekendAIService`	Mistral Large
Achats famille	`AchatsIA.suggestions_enrichies()` + Vinted + LBC	Mistral Large
Projets maison	`ProjetsIAMixin.estimer_budget()`	Mistral Large
Garanties	Prédictions pannes	Mistral Large
Jeux	`JeuxAIService.suggerer_strategies()`	Mistral Large
Loto/Euromillions	Grilles IA pondérées	Mistral Large
Import recettes	Enrichissement nutrition/bio/robots	Mistral Large
Chat général	Multi-contexte (cuisine/maison/famille/jeux)	Mistral Large
OCR tickets	`ticket_caisse.py` (Pixtral)	Pixtral 12B
Résumé hebdo famille	`ServiceResumeHebdo`	Mistral Large
Génération recette depuis photo	`POST /recettes/generer-depuis-photo`	Pixtral 12B
Coaching hebdo Jules	`GET /famille/jules/coaching-hebdo`	Mistral Large
Version recette Jules	`ServiceVersionRecetteJules`	Mistral Large

Module	Service IA	Modèle
WhatsApp Intent → Action	`webhooks_whatsapp.py` (Mistral interprète)	Mistral Large

> **RouteurIA** (`src/core/ai/router.py` , 704 lignes) : nouveau composant de routage multi-fournisseur ajouté en Sprint 5 — `_ClassifieurComplexite` analyse la requête et route automatiquement vers le bon modèle/fournisseur selon la complexité.

Nouveaux axes IA à implémenter

🤖 IA-01 : Assistant vocal (Speech-to-Text → Action)

Concept : Dictaphone dans l'app → transcription Whisper → interprétation Mistral pour effectuer des actions.
Exemples :

- "Ajoute du lait à la liste de courses" → `POST /api/v1/courses/{id}/articles`
- "J'ai pesé Jules, 11,4 kg" → `POST /api/v1/famille/jules/croissance`
- "Rappelle-moi d'appeler le plombier mardi" → `POST /api/v1/maison/routines`

Stack : Web Speech API (navigateur, gratuit) ou `whisper-tiny` local → Mistral pour extraction d'intention → dispatch vers API.
Effort : L — nécessite mapping intention → API.

✅ IA-02 : Génération recette depuis photo plat — IMPLÉMENTÉ

API : `POST /api/v1/recettes/generer-depuis-photo` (multipart image → RecetteResponse) — **opérationnel**.

✅ IA-03 : Résumé hebdo intelligent multi-modules — IMPLÉMENTÉ

Job : `resume_hebdo` (Lundi 07h30) — cron actif. **API manuelle** : `POST /api/v1/admin/jobs/resume_hebdo/run` .

🤖 IA-04 : Planificateur de vacances IA

Concept : "On part en Bretagne 5 jours avec Jules (22 mois)" → IA génère : planning d'activités, liste de courses adaptée, checklist voyage (déjà service `voyage.py`), budget prévisionnel.
API : `POST /api/v1/famille/voyage/planifier-ia` .
Effort : M — services voyage et checklist existent.

🤖 IA-05 : Détection d'anomalies financières inter-modules

Concept : Analyser en cross-module toutes les dépenses (maison + famille + jeux) pour détecter patterns inhabituels ("tes dépenses courses ont augmenté de 35% ce mois").
API : `GET /api/v1/dashboard/anomalies-financieres` .
Effort : M.

✅ IA-06 : Coaching Jules proactif — IMPLÉMENTÉ

API : `GET /api/v1/famille/jules/coaching-hebdo` — **opérationnel**. `JulesAIService` analyse la semaine (activités, repas, carnet santé) → message hebdo contextuel.

🤖 IA-07 : Optimisation budget courses IA

Concept : Analyser l'historique d'achat + les prix habituels → suggérer les substitutions économiques ("Remplace noix de cajou par noix de Grenoble, économie estimée 3€/semaine").
API : `GET /api/v1/courses/optimiser-budget-ia` .
Effort : M.

🤖 IA-08 : Vision pour diagnostic maison

Concept : Uploader une photo d'un problème maison (fissure, humidité, panne) → Pixtral identifie le type de problème → Mistral génère diagnostic + devis estimatif + recommandations artisans.

API : `POST /api/v1/maison/diagnostics/ia-photo` .
Effort : M.

🤖 IA-09 : Score bien-être familial global

Concept : Un indicateur 0-100 calculé chaque semaine à partir de : diversité alimentaire de la semaine, Nutri-Score moyen des repas planifiés, activités sportives (saisie manuelle ou Garmin si connecté). Trend hebdomadaire affiché sur le dashboard.
Formule : diversité alimentaire 40% + Nutri-Score hebdo 30% + activités sportives 30%
API : `GET /api/v1/dashboard/score-bienetre` .
Effort : M.

🤖 IA-10 : Suggestions proactives contextuelles (Push IA)

Concept : À 18h chaque jour, analyser le contexte (planning demain, météo, stock) et envoyer 1 notification personnalisée : "Demain il fait -5°, pense à décongeler le poulet" ou "Jules n'a pas eu d'activité artiste cette semaine".
Canal : ntfy / Web Push.
Effort : M — nécessite orchestrateur de contexte journalier.

9. Jobs automatiques — Planning cron

Jobs existants (10 actifs)

Schedule	Job ID	Action	Statut
Lundi 06h00	`entretien_saisonnier`	Création tâches entretien saisonnières	✅ Actif
Lundi 07h30	`resume_hebdo`	Résumé hebdo famille complet (ntfy + email)	✅ Actif
07h00 quotidien	`rappels_famille`	Anniversaires, jalons, crèche, documents	✅ Actif
08h00 quotidien	`rappels_maison`	Garanties, contrats, entretien	✅ Actif
08h30 quotidien	`rappels_generaux`	Inventaire, alertes générales	✅ Actif
09h00 quotidien	`push_quotidien`	Web Push alertes urgentes + jeux	✅ Actif
09h00 quotidien	`digest_ntfy`	Digest ntfy.sh (tâches + rappels)	✅ Actif
18h00 quotidien	`rappel_courses`	Alerte ntfy.sh articles courses en attente	✅ Actif
18h00 quotidien	`push_contextuel_soir`	Push contextuel soir (planning demain + météo)	✅ Actif
1er du mois 03h00	`enrichissement_catalogues`	Enrichissement IA 4 catalogues JSON	✅ Actif

Jobs à créer

ID	Schedule	Description	Canal notification	Effort
~~J-01~~	~~Lundi 07h30~~	~~Résumé hebdo complet~~	~~ntfy + email~~	✅ IMPLÉMENTÉ (`resume_hebdo`)
J-02	**18h00 quotidien**	**Push contextuel IA amélioré** : "Ce soir tu prépares X (45min)" basé sur planning	ntfy / Web Push	M
J-03	**Dimanche 20h00**	**Génération planning semaine suivante** IA si le planning est vide	ntfy ("Ton planning est vide, génère-le ?")	S
J-04	**06h00 quotidien**	**Alerte péremptions** : produits expirant dans 48h → suggère recettes	ntfy / Web Push	S

ID	Schedule	Description	Canal notification	Effort
~~J-05~~	~~09h00~~	~~digest ntfy~~	~~ntfy~~	✅ IMPLÉMENTÉ (`digest_ntfy`)
~~J-06~~	~~18h00~~	~~Rappel courses urgentes~~	~~ntfy~~	✅ IMPLÉMENTÉ (`rappel_courses`)
J-07	**1er du mois**	**Rapport mensuel budget** : synthèse famille + maison + jeux + recommandations IA	ntfy / email	M
J-08	**Vendredi 17h00**	**Score weekend** : suggestions activités basées météo des 2 prochains jours + état Jules	ntfy	S
J-09	**1er du mois**	**Contrôle garanties et contrats** : rappels expirations dans les 3 prochains mois	ntfy / email	XS (déjà 08h00 maison, à enrichir)
J-10	**Mercredi 20h00**	**Rapport jardin** : arrosage, semis prévus cette semaine selon données météo	ntfy	S
J-11	**Dimanche 20h00**	**Score bien-être hebdo** : calcul diversité alimentaire + Nutri-Score + activités sportives → alerte si dérive	ntfy + dashboard	S

10. Notifications — WhatsApp, Mail, Push

État actuel (Sprint 5)

Canal	Service	Statut	Couverture
ntfy.sh	`ServiceNtfy` (298 lignes)	✅ Opérationnel + schedulé	Digest quotidien 09h00, rappel courses 18h00, maison, tâches
Web Push (VAPID)	`ServiceWebPush` + `NotificationPersistenceMixin`	✅ Opérationnel (DB-persisté)	Famille, jeux, maison, alertes urgentes
Email	`ServiceEmail` via **Resend** (230 lignes)	✅ Opérationnel	Reset password, verify email, résumé hebdo, alertes critiques, invitation famille
WhatsApp	`webhooks_whatsapp.py` (296 lignes)	✅ Webhook entrant actif	Réception messages + réponse IA via Meta Cloud API
Dispatcher	`DispatcherNotifications` (185 lignes)	✅ Opérationnel	Multi-canal `envoyer(user_id, message, canaux=["push","email","ntfy"])`
SMS	—	❌ Absent	—

Architecture notifications (Sprint 5)

```
src/services/core/notifications/
├── types.py (279L)                 ← Types, VAPID_PUBLIC_KEY, TypeNotification
├── notif_email.py (230L)           ← ✅ ServiceEmail (Resend) — 6 méthodes
├── notif_ntfy.py (298L)            ← ✅ ServiceNtfy + PlanificateurNtfy
├── notif_web_core.py (293L)        ← ✅ ServiceWebPush (pywebpush)
├── notif_web_persistence.py (317L) ← ✅ NotificationPersistenceMixin (DB)
├── notif_web_templates.py (235L)   ← ✅ Templates par domaine (famille, jeux, maison)
├── notif_dispatcher.py (185L)      ← ✅ DispatcherNotifications multi-canal
├── inventaire.py (278L)            ← ✅ ServiceNotificationsInventaire
└── utils.py (567L)                 ← Helpers mapping types
```

Canal Email — méthodes disponibles (`ServiceEmail`)

```
service_email.envoyer_reset_password(email, token)
service_email.envoyer_verification_email(email, token)
service_email.envoyer_resume_hebdo(email, resume_data)
service_email.envoyer_rapport_mensuel(email, rapport_data)
service_email.envoyer_alerte_critique(email, message)
service_email.envoyer_invitation_famille(email, invitant)
```

Canal WhatsApp — webhook entrant

`POST /api/v1/whatsapp/webhook` (Meta Cloud API) :

1. Réception message texte WhatsApp
2. Mistral interprète l'intention ("Ajoute du lait aux courses" → `POST /courses/{id}/articles`)
3. Dispatch vers API interne correspondante
4. Confirmation envoyée en réponse WhatsApp

WhatsApp sortant (proactif) : Pas encore implémenté — prévoir via Meta Cloud API `POST /messages` .

Actions restantes notifications

- **WhatsApp sortant proactif** : messages préemptifs (rappels crèche, liste courses partagée) — Effort M
- **Canal SMS** : optionnel (Twilio / Vonage) — Effort M
- **B-09** : compléter `manifest.json` avec `gcm_sender_id` / `background_sync` pour Chrome Android

11. Mode Admin / Déclenchement manuel

✅ Implémenté (Sprint 5) — `admin.py` 425 lignes

Le panneau admin complet est désormais opérationnel avec `role == "admin"` requis sur tous les endpoints.

Endpoints backend implémentés

Endpoint	Description
`GET /admin/audit-logs`	Liste des actions avec filtres
`GET /admin/audit-stats`	Stats audit (top users, actions, périodes)
`GET /admin/audit-export-csv`	Export CSV de l'audit log
`GET /admin/jobs`	Liste tous les jobs APScheduler + dernier run + statut
`POST /admin/jobs/{job_id}/run`	Déclenchement immédiat d'un job
`POST /admin/notifications/test`	Test envoi notif (canal: ntfy/push/email)
`GET /admin/cache/stats`	Stats cache (hits/misses par couche)
`POST /admin/cache/purge`	Purge cache par pattern
`GET /admin/users`	Liste des comptes + rôles + dernier login

Jobs disponibles dans le panneau admin

ID	Libellé affiché	Catégorie
`rappels_famille`	Rappels famille (anniversaires, crèche...)	Famille
`rappels_maison`	Rappels maison (garanties, contrats...)	Maison
`rappels_generaux`	Rappels généraux (inventaire, alertes)	Général
`push_quotidien`	Push quotidien (alertes urgentes)	Notifications
`digest_ntfy`	Digest ntfy quotidien	Notifications
`rappel_courses`	Rappel courses urgentes	Courses
`entretien_saisonnier`	Entretien saisonnier	Maison
`resume_hebdo`	Résumé hebdomadaire famille	Famille
`push_contextuel_soir`	Push contextuel soir	Notifications
`enrichissement_catalogues`	Enrichissement catalogues IA	IA

Ce qui reste à faire (admin)

- **Page frontend jobs** : `frontend/src/app/(app)/admin/jobs/page.tsx` — UI avec boutons "Exécuter" + badge dernier run + statut (backend prêt, UI non créée)
- **Modification/suppression utilisateurs** depuis l'UI (PATCH/DELETE sur comptes)
- **Tableau monitoring** : visualisation graphes Prometheus dans l'UI admin

12. Axes d'innovation identifiés

🚀 Innovations haute valeur

Innovation-01 : Écosystème d'automations type "IFTTT"

Concept : Interface visuelle "Si [déclencheur] → alors [action]" configurable.
Exemples :

- "Si stock lait < 2 → Ajouter 6 laits aux courses"
- "Si demain > 3°C → Rappel arroser jardin"
- "Si pari gagné > 50€ → Ajouter à budget loisirs"
- "Si Jules a médecin → Ajouter en absent à la crèche"

Stack : Table `automations` en DB + moteur de règles simplifié + APScheduler.
Effort : L mais très haute valeur.

Innovation-02 : Timeline de vie familiale

Concept : Visualisation chronologique de tous les événements importants : naissance Jules, premiers pas, anniversaires, rénovations maison, voyages, victoires sportives.
Source : Jalons Jules + Événements familiaux + Projets maison + Matchs mémorables.
Effort : M.

Innovation-03 : Budget prédictif annuel

Concept : Basé sur l'historique, prédire les dépenses des 12 prochains mois et alerter sur les gros postes à venir (vacances, rentrée scolaire, révision chaudière...).

Effort : M.

Innovation-04 : Mode "Voyage" avec checklists intelligentes

Concept : Déclarer "On part 5 jours à la mer en été" → 4 listes distinctes selon contexte (🏖️ Mer été / ⛷️ Montagne hiver / 🥾 Montagne été / 🌊 Mer hiver), générées depuis le template adapté. Courses sur place au quotidien, pas de cuisine anticipée.
Mécanique d'apprentissage : bouton "Acheté sur place" pendant le séjour → à la fin (date retour ou bouton "On est rentrés 🏠"), popup proposant d'ajouter définitivement les achats imprévus à la liste. Chaque item a un champ `source: "template" | "utilisateur" | "expérience"` tracé.
Désactivation contextuelle : rappels arrosage jardin suspendus pendant la période, planning cuisine allégé, rappels maison en pause.
Effort : M — service `voyage.py` et checklists existent.

Innovation-05 : Connexion Garmin / santé adultes

Concept : Les modèles Garmin existent (`GarminToken` , `ResumeQuotidienGarmin`). Activer l'intégration pour : calories brûlées → recommandations repas du soir ; qualité sommeil → conseil récupération Jules.
Effort : L (OAuth Garmin Connect + webhook daily sync).

Innovation-06 : Intégration Garmin (santé sport)

Concept : Les modèles Garmin existent (`GarminToken` , `ResumeQuotidienGarmin`). Activer l'intégration pour : calories brûlées → recommandations repas du soir ; qualité sommeil → conseil récupération ; activités enregistrées → contribution au score bien-être (IA-09).
Effort : L (OAuth Garmin Connect + webhook daily sync).

Innovation-07 : Gamification sport et alimentation

Concept : Deux axes uniquement : **sport** (points pour chaque activité physique saisie, défis hebdo, streaks) et **mieux manger** (badges Nutri-Score moyen de la semaine, diversité alimentaire, "Zéro gaspi cette semaine"). Tableau de bord points famille sur le dashboard.
Effort : M — `ServiceGamification` existe dans `src/services/core/gamification.py` .

Innovation-08 : Scan reçus et factures automatique

Concept : Uploader une photo de ticket de caisse → Pixtral extrait : montant, date, catégorie, articles → intégration automatique dans :

- Budget famille si supermarché
- Garanties si achat électroménager
- Inventaire si produits reconnus

Déjà partiel : scan ticket courses (OCR Pixtral) existe, à étendre.

💡 Petites innovations rapides (Quick wins)

#	Idée	Effort	Impact
QW-01	Widget météo sur dashboard (déjà API météo existante)	XS	⭐⭐⭐
QW-02	Bouton "Recette Surprise" aléatoire filtrée par saison + frigo	XS	⭐⭐⭐
QW-03	Partage liste de courses par QR code (lien temporaire)	S	⭐⭐⭐
QW-04	Compteur de jours depuis dernière cuisine de chaque recette	XS	⭐⭐
QW-05	Estimation temps de préparation en fonction du niveau de la recette	XS	⭐⭐
QW-06	"Aujourd'hui dans l'histoire de la famille" (événements passés ce jour)	S	⭐⭐⭐
QW-07	Mode sombre/clair forcé selon heure (auto après 21h)	XS	⭐⭐
QW-08	Impression optimisée recette (CSS print)	XS	⭐⭐
QW-09	Export iCal générique (Google Calendar, Android, Thunderbird…) — iCal déjà implémenté	XS	⭐⭐
QW-10	Score anti-gaspi hebdo partageable (image générée)	S	⭐⭐

13. Synthèse & roadmap recommandée

Priorités immédiates (< 1 semaine)

#	Action	Effort	Catégorie
P-01	**Fixer B-01** : persistance push subscriptions en DB	S	🔴 Bug critique
P-02	**Fixer B-02** : endpoint `GET /push/vapid-public-key`	XS	🔴 Bug critique
P-03	**Fixer B-03** : scheduler ntfy digest + rappel courses	XS	🟠 Bug haut
P-04	**SQL-01** : supprimer table `liste_cours` (doublon)	XS	🟠 SQL
P-05	**SQL-05** : déplacer inline ALTER TABLE en tête de CREATE TABLE	S	🟠 SQL
P-06	**SQL-CONSOLIDATION** : absorber `sql/migrations/001	002	003.sql` dans INIT_COMPLET + supprimer les 3 fichiers
P-07	**SQL-CONSOLIDATION** : absorber `alembic/versions/` (baseline vide + refonte famille + anniversaires) dans INIT_COMPLET + supprimer les 3 fichiers	S	🟠 SQL

Court terme (1-3 semaines)

#	Action	Effort	Catégorie
CT-01	**Canal email** : service `ServiceEmail` + reset password + verify	L	📧 Notifications
CT-02	**Mode Admin étendu** : panneau jobs + trigger manuel	M	🔧 Admin
CT-03	**Job J-02** : push contextuel soir (planning demain + météo)	M	⏱️ Cron
CT-04	**Job J-01** : résumé hebdo lundi matin	M	⏱️ Cron
CT-05	**IA-06** : coaching hebdo Jules ()	S	🤖 IA
CT-06	**IA-02** : recette depuis photo (variation Pixtral)	S	🤖 IA
CT-07	**F-01** (Famille) : tests achats + garde	M	✅ Tests
CT-08	**SQL-06** : index manquants sur relations fréquentes	S	SQL
CT-09	**Bouton Version Jules** sur fiche recette + profil aliments exclus	S	👶 Famille/Cuisine
CT-10	**SQL-03/04** : audit et nettoyage tables orphelines ORM↔SQL (`voyage.py` sans CREATE TABLE, `journal_bord` / `sessions_travail` sans ORM)	M	🟠 SQL

#	Action	Effort	Catégorie
CT-11	**Doc SQL** : mettre à jour `docs/MIGRATION_GUIDE.md` (retirer workflow SQL-file, documenter INIT_COMPLET only) + archiver `sql/migrations/README.md` + `alembic/README.md`	S	📚 Doc
CT-12	**Test schéma** : créer `tests/sql/test_schema_coherence.py` — vérifie automatiquement que chaque `__tablename__` ORM a un `CREATE TABLE` correspondant dans INIT_COMPLET.sql	S	✅ Tests
CT-13	**Tests push** : créer `tests/api/test_push_notifications.py` — couvre B-01 (persistance abonnements), B-02 (endpoint VAPID), B-03 (scheduler ntfy/courses)	M	✅ Tests
CT-14	**Tests routes** : compléter couverture admin (`/admin/users` , `/admin/jobs`), recherche globale, RGPD (`/admin/export-data` , `/admin/delete-account`)	M	✅ Tests
CT-15	**Supprimer `STATUS_PHASES.md`** : 1004 lignes désormais redondantes avec `ANALYSE_COMPLETE.md` . Retirer toutes les références (`ROADMAP.md` section "Système de Phases", `docs/INDEX.md`). `ANALYSE_COMPLETE.md` devient la référence unique d'état du projet	S	📚 Doc
CT-16	**Néttoyer `ROADMAP.md`** : supprimer l'historique des sprints déjà livrés (sections "Mise à jour copilot-worktree", "Sprint 2", "Sprint 3" etc.), retirer la section "Système de Phases A-AC" (absorbée par ANALYSE_COMPLETE.md), aligner la table "Priorités 2 semaines" sur la section 13 de ANALYSE_COMPLETE.md	M	📚 Doc
CT-17	**Nettoyage `docs/`** : supprimer `JEUX_AMELIORATIONS_PLAN.md` + `JEUX_PLAN_VALIDATION.md` (jeux 100% complet, obsolètes), `docs/guides/batch_cooking.md` (fichier vide 0 ligne), `markdown-preview.md` (fichier de debug) ; mettre à jour `docs/INDEX.md` (retirer lien `STATUS_PHASES.md` , ajouter lien `ANALYSE_COMPLETE.md` comme référence principale)	S	📚 Doc

Moyen terme (1-2 mois)

#	Action	Effort	Catégorie
MT-01	**Interaction** Cellier ↔ Inventaire cuisine	M	🔗 Inter-modules
MT-02	**WhatsApp sortant proactif** (envoi messages, rappels préemptifs)	M	📱 Notifications
MT-03	**Dashboard score bien-être** (IA-09 : sport + alimentation)	M	📊 Dashboard
MT-04	**Page frontend jobs admin** : UI boutons déclenchement + badges statut	S	🔧 Admin
MT-05	**Innovation-01** : Automations "Si → Alors"	L	🚀 Innovation
MT-06	**Widgets dashboard configurables**	L	📊 Dashboard
MT-07	**IA-01** : Assistant vocal (Speech-to-Text → Action)	L	🤖 IA
MT-08	**Innovation-02** : Timeline vie familiale	M	🚀 Innovation
MT-09	**OCR photo-frigo** auto-sync inventaire (Phase K complète)	S	🍽️ Cuisine

Long terme (3+ mois)

#	Action	Effort	Catégorie
LT-01	**Innovation-06** : Intégration Garmin santé sport	L	🚀
LT-02	**Innovation-07** : Gamification sport + alimentation	M	🚀
LT-03	**Innovation-04** : Mode Voyage with checklists intelligentes	M	🚀
LT-04	**Innovation-01** : Automations “Si → Alors”	L	🚀

Annexe A — Résumé des alertes SQL

```sql
-- Doublon tables à nettoyer
DROP TABLE IF EXISTS liste_cours;  -- garder listes_courses

-- Index manquants à ajouter dans INIT_COMPLET.sql
CREATE INDEX IF NOT EXISTS idx_recette_ingredients_ingredient_id ON recette_ingredients(ingredient_id);
CREATE INDEX IF NOT EXISTS idx_repas_planning_id ON repas(planning_id);
CREATE INDEX IF NOT EXISTS idx_articles_courses_liste_id ON articles_courses(liste_id);
CREATE INDEX IF NOT EXISTS idx_historique_inventaire_article_id ON historique_inventaire(article_id);
CREATE INDEX IF NOT EXISTS idx_jalons_profil_id ON jalons(profil_enfant_id);
CREATE INDEX IF NOT EXISTS idx_paris_user_date ON jeux_paris_sportifs(user_id, date_pari);

-- Table Push Subscriptions manquante
CREATE TABLE IF NOT EXISTS abonnements_push_vapid (
    id BIGSERIAL PRIMARY KEY,
    user_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
    endpoint TEXT UNIQUE NOT NULL,
    p256dh TEXT NOT NULL,
    auth_key TEXT NOT NULL,
    user_agent TEXT,
    actif BOOLEAN DEFAULT TRUE,
    cree_le TIMESTAMPTZ DEFAULT NOW(),
    dernier_ping TIMESTAMPTZ
);
ALTER TABLE abonnements_push_vapid ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Utilisateur voit ses abonnements" ON abonnements_push_vapid
    FOR ALL USING (auth.uid() = user_id);
```

Annexe B — Résumé des fichiers à créer

Fichier	Contenu	Statut
~~`src/services/core/notifications/notif_email.py`~~	~~`ServiceEmail` avec Resend~~	✅ FAIT (Sprint 5)
~~`src/services/core/notifications/notif_dispatcher.py`~~	~~Dispatcher multi-canal~~	✅ FAIT (Sprint 5)
~~`src/api/routes/admin.py` (enrichi)~~	~~+endpoints jobs, notif test, cache, users~~	✅ FAIT (Sprint 5)
~~`src/services/famille/version_recette_jules.py`~~	~~Génération VersionRecette Jules~~	✅ FAIT (Sprint 5)
`src/services/core/notifications/notif_whatsapp.py`	Client WhatsApp sortant proactif	🟡 MT-02
`src/services/core/automations/`	Moteur "Si → Alors"	🟡 MT-05
`frontend/src/app/(app)/admin/jobs/page.tsx`	UI déclenchement manuel des jobs	🔴 MT-04
`tests/sql/test_schema_coherence.py`	Vérifie cohérence ORM ↔ INIT_COMPLET.sql	🔴 CT-12
`tests/api/test_push_notifications.py`	Tests persistance abonnements, endpoint VAPID, scheduler	🔴 CT-13
`tests/api/test_admin_routes.py`	Couverture `/admin/users`, `/admin/jobs`, export RGPD, suppression compte	🟡 CT-14

Fichiers à supprimer (CT-15/CT-16/CT-17)

Fichier	Raison	Priorité
`STATUS_PHASES.md` (racine)	1004 lignes redondantes avec ANALYSE_COMPLETE.md	🟠 CT-15
`docs/JEUX_AMELIORATIONS_PLAN.md`	Module Jeux 100% complet — plan obsolète	🟠 CT-17
`docs/JEUX_PLAN_VALIDATION.md`	Module Jeux 100% complet — plan de validation obsolète	🟠 CT-17
`docs/guides/batch_cooking.md`	Fichier vide (0 ligne)	🟠 CT-17
`docs/markdown-preview.md`	Fichier de debug/preview (17 lignes)	🟠 CT-17

Rapport généré le 27 mars 2026 — Mis à jour le 29 mars 2026 (Sprint 5 : notifications, admin, 2FA, Version Jules) — GitHub Copilot